

Large-Scale Collective Entity Matching

Vibhor Rastogi
University of Washington

Nilesh Dalvi
Yahoo! Research

Minos Garofalakis
Technical University of Crete

ABSTRACT

The difficult problem of *Entity Matching (EM)*, i.e., determining whether different mentions of an entity refer to the same real-world object, has recently attracted a lot of attention from the Machine Learning (ML) research community. State-of-the-art solutions for EM are based on recent advances in ML, such as first-order probabilistic models (e.g., Markov and Bayesian networks) and advanced probabilistic inference techniques. A key benefit of these ML tools comes from their *purely collective* nature: match evidence for related entities can be collectively reinforced into high-probability EM decisions. In addition, they provide a principled framework for imposing a probability distribution over possible EM results which could be useful in many settings (e.g., for user feedback). While such state-of-the-art ML approaches to EM have been shown to be very accurate in practice, they also typically require complex inference over very large model graphs; thus, their scalability to real-life datasets has remained a big challenge. Toward this end, in this work, we propose a principled framework to scale general, collective EM operators. Our framework is generic: it uses a black-box abstraction to incorporate any entity matcher. The main idea is to approximate the run of the entity matcher on the entire data set by: (1) running multiple instances of the matcher on several small subsets of the entities, and (2) *Message-Passing*, i.e., passing a judiciously-built message-set across the instances to control the interaction between different runs of the matcher. While the notion of communicating “blocks” of EM is not entirely new, our work is the first to carry out a complete, formal analysis of the above framework, and show that for a broad class of “well-behaved” entity matchers, the approach is *provably sound*. We also propose novel message-passing schemes for probabilistic EM tools that, as our results demonstrate, significantly improve EM recall without compromising soundness. Finally, we present experimental results demonstrating the effectiveness of our approach and its ability to scale to large real-life datasets.

1. INTRODUCTION

Entity Matching (EM) refers the problem of determining whether two entities in a data set refer to the same real-world object. Entity matching is a complex and ubiquitous problem that appears in numerous application domains (including image processing, information extraction and integration, and natural language processing), often under dif-

ferent terminology (e.g., coreference resolution, record linkage, and deduplication). While early work on EM dates back to the 1960s [18, 9], the problem has recently received renewed interest due mainly to the growing popularity of data exchange and sharing (especially, in the science community), and advances in technologies for web-scale information extraction and management.

Conventional approaches to EM (dating back to the seminal work of Newcombe [18] and Fellegi and Sunter [9]) have focused on discovering independent pair-wise matches of entities using a variety of attribute-similarity measures (based, for instance, on approximate string matching). Recently, more general EM methodologies have been proposed that also take into account *structural (i.e., relational)* information about the entity set. For instance, in a paper-bibliography database, paper co-authorship and citation information can be exploited when matching two author entities. *Relational EM* approaches simply look at the attributes of related entities and incorporate them in the computation of the attribute-similarity scores (which are still computed in a pair-wise fashion). More sophisticated, *Collective EM* methods also exploit relational information but do not depend on making independent pair-wise decisions; instead, they explicitly model the interrelationships of related matching decisions, and simultaneously decide which entities (e.g., authors and papers) to match in a truly-collective fashion. State-of-the-art Collective EM methods typically rely on sophisticated Machine Learning (ML) tools: The idea is to (1) model the EM problem using advanced *probabilistic models* (to capture the interrelationships across EM decisions, represented as random variables in the model), and (2) express the EM task in terms of *probabilistic inference* operations over the model. Several probabilistic models have recently been exploited for Collective EM tasks, including relational Bayesian and Markov networks [21, 19], Conditional Random Fields (CRFs) [16, 6], and latent Dirichlet models [3, 11]. Probabilistic models and inference provide a clean and principled framework for Collective EM, and have demonstrated significant accuracy benefits for practical EM problems; in addition, they naturally define a probability distribution over possible EM results which could be useful in many settings (e.g., for user feedback). Unfortunately, probabilistic EM tools also suffer from a fundamental problem, namely, their poor scalability: the high cost of probabilistic inference over large EM graphs renders these methods computationally infeasible for large data sets.

Of course, the issue of scalability arises even in the simple case of independent pair-wise entity matchers, due to the

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the VLDB copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Very Large Data Base Endowment. To copy otherwise, or to republish, to post on servers or to redistribute to lists, requires a fee and/or special permission from the publisher, ACM.

VLDB '09, August 24-28, 2009, Lyon, France

Copyright 2009 VLDB Endowment, ACM 000-0-00000-000-0/00/00.

obviously quadratic complexity of the all-pairs comparisons. This problem has been addressed in the literature using techniques that rely on *blocking* [20, 15]. Blocking methods try to group entities together based on simple, heuristic grouping criteria (e.g., by the initial letter of authors’ last names), so that matching entities are very likely to fall under the same group. Instances of the EM algorithm are then run on each group individually and matching results are collected across all the groups. In general, blocking groups can overlap, and several different grouping criteria can be used (to increase the probability that matching entities are paired-up in some EM instance). For independent pair-wise entity matchers, blocking helps avoid the quadratic cost of all-pairs comparisons, essentially reducing the complexity of EM on n entities from $O(n^2)$ to $O(kn)$, where $k \ll n$ denotes the average size of a group [15]. More recently, Whang et al. [8] have proposed an *iterative blocking* framework, where the key idea is to allow blocks to communicate their “local” EM results in an iterative manner (until a fixpoint is reached). As they demonstrate, sharing local matches allows for both better accuracy (by enabling some collective EM decisions across groups) and better runtimes (by avoiding duplicated matching effort); still, their development is, for the most part based on heuristics and does not provide any formal guarantees on the quality of the EM results. In addition, [8] does not consider the more complex case of *probabilistic* Collective EM, that, as discussed earlier, significantly exacerbates the scalability issues involved.

Our Contributions. In this paper, we propose a principled framework for scaling state-of-the-art, Collective EM algorithms to large data sets. Our framework allows for general EM algorithms modeled as “blackboxes” that take in a set of entities along with a collection of *evidence*, and output a set of matches. Our framework approximates the run of the matcher on the entire data set by: (1) running multiple instances of the matcher on several small subsets of the entities, and (2) *Message-Passing*, i.e., passing a judiciously-built message-set across the instances to exchange information between different runs of the matcher. We present a formal analysis of our framework, demonstrating that, for a broad class of “well-behaved” entity matchers (satisfying natural “monotonicity” properties), the approach is *provably sound*. We also consider the case of Collective EM based on probabilistic models, and propose novel message-passing schemes that significantly improve EM recall without compromising soundness. The salient properties of our EM framework can thus be summarized as follows.

1. *Generic*: Any (deterministic or probabilistic) EM algorithm can be incorporated in our framework.
2. *Accurate & Provably Sound*: Our formal analysis demonstrates soundness for a broad class of EM techniques, while novel message-passing schemes guarantee high recall.
3. *Scalable & Parallelizable*: By running EM instances on small subsets of entities together with intelligent message passing, our framework is highly scalable and also suitable for a parallel implementation.

Our experimental study over real data sets verifies the effectiveness of our approach, demonstrating the ability to scale a state-of-the-art Collective EM algorithm based on Markov

Logic Network [21] to real-life data sets of more than 58,000 references involving around 1.3 million entity pair matching decisions with little or no impact on the accuracy of the results.

2. THE ENTITY MATCHING PROBLEM

In this section, we formalize the Entity Matching (EM) problem and give a brief survey of existing EM techniques examining their accuracy/efficiency trade-offs.

ENTITY MATCHING (EM) PROBLEM : *Let E denote a collection of entity mentions. Each element e in E is characterized by a set of attributes, $e.A_1, e.A_2, \dots, e.A_k$ (some of which may be undefined). Furthermore, we have a set of relations, $\mathcal{R} = R_1, \dots, R_m$ over E . The objective is to determine, given $e_i, e_j \in E$, whether e_i and e_j refer to the same entity in real world.*

As an example, consider the following EM instance — this will be our running example throughout the paper.

EXAMPLE 1. *Consider a collection of paper publications obtained from multiple bibliography databases. Our goal is to determine which paper (author) mentions in the database refer to the same actual paper (resp., author). Here, E is the set of all paper and author mentions in our database. Each paper mention $p \in E$ has a subset of the attributes $p.title, p.journal, p.year, p.category$, while each author $a \in E$ has attributes $a.fname, a.lname$. We further have three relations defined over E : $\mathcal{R} = \{\text{Authored}, \text{Cites}, \text{Coauthor}\}$, where $\text{Authored}(a, p)$ denotes that a is an author of paper p , $\text{Cites}(p_1, p_2)$ states that paper p_1 cites paper p_2 , and $\text{Coauthor}(a_1, a_2)$ denotes that authors a_1 and a_2 have co-authored some paper. (Note that the *Coauthor* relation can actually be easily derived through a self-join on *Authored*.)*

To simplify our discussion, in the remainder of the paper, we omit the term “mentions” and refer to elements of E simply as “entities”; thus, our goal is to determine *matching entities* (i.e., entities representing the same real-world object). Existing EM work can be roughly classified in three broad categories.

- *Non-Relational Approaches*: Initial approaches to EM focused on pair-wise attribute similarities between entities. Newcombe [18] and Fellegi and Sunter [9], gave the problem a probabilistic foundation by posing EM as a classification problem (i.e., deciding a pair to be a match or a non-match) based on attribute-similarity scores. The bulk of follow up work on EM then focused on constructing good attribute-similarity measures (e.g., using approximate string-matching techniques) [17, 5, 2]. For instance, in Example 1, an attribute-based matcher decides to match two authors by estimating the similarity of their *fname* and *lname* strings. A fundamental shortcoming of attribute-based matchers is that they cannot perform disambiguation, e.g., if there are two different authors with the same name “J. Doe”, they cannot be identified as separate entities.

- *Simple Relational Approaches*: Entity matching can be significantly improved by using relational information in addition to attribute similarities. Simple relational techniques take the attributes of *related entities* into account when computing the similarity scores of entity pairs [1, 12]. For

instance, in Example 1, when attempting to match two authors, looking at the attribute values of their co-authors and the set of authors that they cite can improve the prediction accuracy by a large margin. For example, two authors “J. Doe” and “John Doe” are more likely to be the same person if they have written papers with “M. Smith” and “Mark Smith”, respectively. While still making matching decisions in a pair-wise manner, the use of relational information can help alleviate the problem of disambiguation. However, one potential pitfall is that “M. Smith” and “Mark Smith” might actually be two different authors with similar names.

- *Collective Approaches*: Collective approaches to EM make use of the fact that various matching decisions are interrelated. For instance, in the scenario described above, the match decision for “J. Doe” and “John Doe” is closely tied to the match decision for “M. Smith” and “Mark Smith”. Thus, collective approaches do not just look at additional attribute values obtained through relational information; instead, they exploit *related match decisions* when trying to determine a given entity match.

Collective EM techniques can be further classified into two sub-classes, *iterative* and *purely-collective* approaches. As their name suggests, iterative approaches [4, 7] iterate over the set of current matches, and, as match decisions are made, they are used to trigger further match decisions. For instance, if there is strong evidence to match “M. Smith” and “Mark Smith”, this decision in turn can be used to match “J. Doe” and “John Doe” by exploiting the co-authorship relation. While intuitively simple, iterative approaches have the problem of *bootstrapping*: if we initially start from an empty set of matches, we might never have enough evidence to perform any match. On the other hand, purely-collective approaches avoid the bootstrapping problem by building sophisticated models of the interrelationships of related match decisions, and using these models to make match decisions in a truly-collective manner. For example, while neither of the (“M. Smith”, “Mark Smith”) and (“J. Doe”, “John Doe”) pairs might have strong evidence by itself to be declared a match, a purely-collective matcher may be able to declare both of them as matches collectively, due to their mutually-reinforcing relation (through co-authorship). Purely-collective approaches represent the current state-of-the-art in terms of matching quality (i.e., the accuracy of matches), and are typically based on recent ML advances in statistical relational learning. Such state-of-the-art EM tools rely on various various advanced probabilistic models, including Conditional Random Fields (CRFs) [16, 6], relational Bayesian networks [19], latent Dirichlet models [3, 11], and Markov Logic Networks [21]. Probabilistic models and inference provide a clean and principled way to model relational information and perform purely-collective EM; unfortunately, the high cost of probabilistic inference over large EM models has hitherto rendered such methods infeasible for large data sets.

3. OUR SCALABLE EM FRAMEWORK : OVERVIEW

In this section, we give an overview of our framework for scaling collective entity matchers. Our basic methodology is to approximate the run of the entity matcher on the entire dataset by (i) running multiple instances of the matcher on several smaller fragments of the database, and (ii) passing

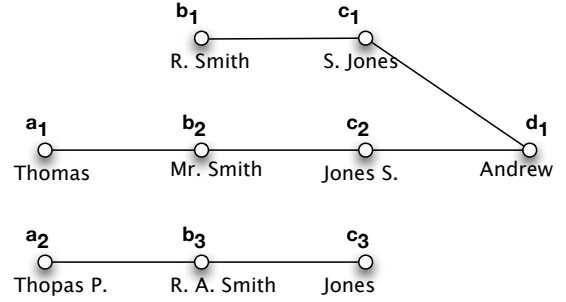


Figure 1: An Instance of Entity Matching Problem: Nodes represent author references and edges represent coauthor relationships.

messages across the instances to simulate the collective nature of the matcher. To illustrate our approach, we first take a look inside one such collective entity matcher. We pick the *Markov Logic Network (MLN)*-based entity matcher [21], which is a state-of-the-art purely-collective EM technique.

Section 3.1 gives an intuitive introduction to MLN matchers, describing how such a matcher works on a simple example. In Section 3.2, we use this MLN matcher to explain informally our scaling framework. Then, in Sections 4–6, we give the formal descriptions of the various components of our framework in more detail.

3.1 The MLN Entity Matcher

Consider a set of entities given in Figure 1, where each entity represents an author reference, and an edge between two entities represent a *Coauthor* relationship. Thus, we have *Coauthor*(a_1, b_2), *Coauthor*(a_2, b_3) etc. Further, using a string-similarity function, we define a relation called *Similar* on the entities. In this example, assume that we have *Similar*(a_1, a_2), *Similar*(b_i, b_j) and *Similar*(c_i, c_j) for $i, j = 1, 2, 3$.

We want to use the relations *Similar* and *Coauthor* to derive a relation *Match*, which tells us the entities that should be matched. Intuitively, we want to match two entities if they are similar and they have a matching coauthor. We can express this as the following two rules:

$$\begin{aligned} R_1 &: \text{Similar}(x, y) \Rightarrow \text{Match}(x, y) \\ R_2 &: \text{Similar}(x_1, y_1) \wedge \text{Coauthor}(x_1, x_2) \wedge \text{Coauthor}(y_1, y_2) \\ &\quad \wedge \text{Match}(x_2, y_2) \Rightarrow \text{Match}(x_1, y_1) \end{aligned}$$

Note that these rules are *not hard constraints*, but rules which we believe are likely to hold. In the MLN matcher, one can write arbitrary rules and the system assigns a weight (a number) to each one of them. (These MLN rule weights are typically learned by using (labeled) training data.) The weights are used to assign a score to each possible set of entity matches: the score of a set is given by the *total weight* of all the rules in that set that become true. Although we will not go into the technical details of MLNs, it is important to note that these MLN rule weights have a natural probabilistic interpretation, and the score assigned to a set of matches can be normalized to obtain a valid probability measure over possible match decisions [21]. Thus, the system tries to find the *highest-scoring set of entity matches* (which is also the

set of matches with the highest probability).

As an example, let us assume that here the system assigns a weight of -5 to R_1 and a weight of 8 to R_2 . A negative weight for R_1 implies that the *Similar* relation by itself is not sufficient to match the two entities. To see this, consider the entities c_1, c_2 for which $\text{Similar}(c_1, c_2)$ holds; hence, the implicant (left-hand side) of R_1 becomes true for c_1 and c_2 . We consider two possible sets of matches: (i) the set $\{\text{Match}(c_1, c_2)\}$, and (ii) the empty set $\{\}$. For the first set, R_1 becomes true and hence the set is assigned a score of -5, while for the second set, R_1 becomes false and hence the set is assigned a weight of 0. Thus, if only rule R_1 was present, we would always prefer the second set over the first and declare c_1 and c_2 to be non-matches.

Now, consider both rules R_1 and R_2 , and again compare the scores for the two sets. For the first set, R_1 is true (as discussed above) and R_2 is also true (by putting $x_2 = y_2 = d_1$)¹. Thus, the score assigned to this set is $-5 + 8 = 3$. On the other hand, the second (empty) set makes R_1 and R_2 both false, and is assigned a score of 0. So, the first set is assigned a larger score, and the system declares c_1 and c_2 to be matches.

In the above deduction, just by looking at c_1, c_2 , and d_1 , we decided that assigning $\text{Match}(c_1, c_2)$ leads to optimal solution. But, how do other entities in the graph affect this decision? One of the nice properties of our MLN-based matcher is that $\text{Match}(c_1, c_2)$ is still the right decision in the globally-optimal solution, even as more entities and relationships are added to the graph. We call this the *monotonicity* property (formally defined in Sec. 4). It turns out that monotonicity is a general property that real-life entity matchers often exhibit, and our EM framework is designed to exploit this.

Continuing with our example in Figure 1, having matched c_1, c_2 , the matcher also infers $\text{Match}(b_1, b_2)$ for similar reasons, since the match further increases the total weight by $8 - 5 = 3$. At this point, we cannot further increase the total weight of the solution by adding any new single match. However, the pairs (a_1, a_2) , (b_1, b_2) and (c_1, c_2) show an interesting property, which truly illustrates the collective nature of MLN. Setting any one of them as a match just adds a weight of +5; but, setting all three of them *collectively* increases the total weight, as R_1 fires three times (once on each pair) leading to a weight of -15 but R_2 also fires two times contributing 16. The net result is an increase in total weight by +1. Thus, our MLN matcher decides to match all three pairs.

In general, MLN-based matchers consider all matching decisions collectively, and there could be large collections of entity pairs, where *matching all the pairs* in the set leads to an increase in the total score, but the advantage can only be seen when looking at the set of pairs as a whole (as in the example above).

3.2 Our Framework

We now give an overview of our scalable EM framework. In a nutshell, our approach consists of three key components: (i) modeling an entity matcher as a *black box* (ii) running multiple instances of the matcher on small subsets of the entities (termed *covers*), and (iii) using *message passing* across the instances to control the interaction between

¹Our MLN also specifies the trivial *reflexivity* rule (i.e., each entity is equal to itself), so that $\text{Match}(d_1, d_1)$ holds.

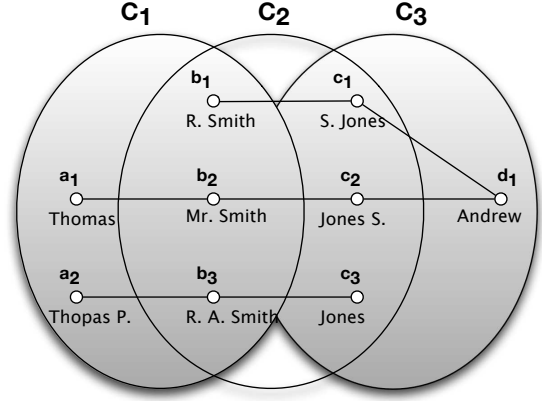


Figure 2: A Covering of Entities.

different runs of the matcher. Below, we present these three key components informally, and also outline some key theoretical properties of our framework.

Black-box EM Abstraction. We consider a very generic abstraction for an entity matcher that takes a set of entities and outputs a set of matches. We define two kinds of matchers : a *deterministic matcher* that simply outputs a set of matches, and a *probabilistic matcher* that also outputs a set of matches, but uses a complete probability distribution over the set of matches to do so. (Formal definitions are in Sec. 4.) We also list a set of desirable properties of an entity matcher, which we can exploit in our framework. We call a matcher *well-behaved* if it satisfies these properties.

The deterministic abstraction is very general and can model any entity matcher. The probabilistic abstraction is a special case of a deterministic matcher, where the final output of the probabilistic matcher is the set of matches that has the highest probability. The reason for defining the probabilistic abstraction as a special case is that we can apply additional techniques when we know that the matcher is backed by a probability distribution. All known, state-of-the-art purely-collective matchers (which are the main focus of our scaling effort), are indeed probabilistic.

Covers. A *cover* is simply a subset of the entities E and a *covering* is a set of (potentially overlapping) covers whose union is E . An example covering of entities in Figure 1 is given in Figure 2: it describes a covering consisting of three covers, C_1, C_2 , and C_3 .

There is a vast amount of literature on *blocking* techniques [20, 15], whose objective is to efficiently group together entities that are similar. The notion of covering extends the notion of blocking to group entities based on not just their similarity, but also on their relational closeness. For instance, in cover C_3 shown in Figure 2, entities d_1 and c_1 are in the same cover not because they are similar, but because they share a coauthorship edge. More generally, a cover may even have entities of different types, e.g. an “*Author*” entity as well as a “*Paper*” entity. Sec. 5 describes covering formally.

Message Passing. We now come to the most important component of the framework : *message passing*. We start

by analyzing how the MLN matcher described in Sec. 3.1 runs when applied separately to each of the three covers in Figure 2. In cover C_1 , MLN will not output any match. This is because none of the pairs (a_1, a_2) and (b_1, b_2) have enough evidence to get matched. If we try to match both pairs, we will incur a weight of +8 by rule R_2 but a weight of -10 by rule R_1 (which fires twice). So, it will decide not to match either of them. Similarly, it will not produce any match in C_2 . In C_3 , it matches the pair (c_1, c_2) as shown in Sec. 3.1. Thus, the total set of matches produced by all the covers is simply $\{(c_1, c_2)\}$.

We notice that MLN misses several matches when run separately on each cover. To recover these misses, we pass *messages* between covers. A *simple message* is just a set of matches found by a cover. For instance, after C_3 runs, it passes a message to the cover C_2 stating $\text{Match}(c_1, c_2)$. When C_2 runs MLN again with the new evidence $\text{Match}(c_1, c_2)$ added to its input, it is now able to match $\text{Match}(b_1, b_2)$. Thus, we recover one of the missed messages. The resulting scheme is called *simple message passing scheme*, denoted by SMP, and is described formally in Sec. 6.

However, the simple message passing scheme cannot recover matches (a_1, a_2) , (b_2, b_3) and (c_2, c_3) . This is because, as we showed earlier, matching the three pairs increases the overall score only if *all three of them* are matched. Thus, neither C_1 nor C_2 alone can match any of the pairs. Furthermore, simple messages are no help as no matches are found in either cover. To overcome this problem, we extend the notion of a simple message. Although C_1 cannot match pairs (b_2, b_3) and (c_2, c_3) , it can deduce that matching any one of them also results in the other pair being matched; thus, either *both pairs* should be matched or *none of the two pairs* should be matched. This is called a *maximal message*. It consists of a set of correlated matches, such that either all of them are true or none of them are. Intuitively, it represents a “*partial inference*” by a cover, waiting to be completed. C_2 thus generates a maximal message $\{(b_2, b_3), (c_2, c_3)\}$. Similarly, C_1 generates a maximal message $\{(a_1, a_2), (b_2, b_3)\}$. These two messages, when combined, essentially “*complete the chain*”, and result in all three pairs being matched. The resulting scheme is termed *maximal message passing scheme*, denoted by MMP, and is developed in detail in Sec. 6.2.

This completes the overview of our framework. Note that each of the three components are independent of one another. Thus, we can choose any entity matcher, any covering technique for dividing data, and combine it with any message passing scheme for exchanging information across covers.

3.2.1 Theoretical Properties

We now describe some key properties of the framework that we want to analyze, theoretically and empirically. Let $\mathcal{E}$ denote an entity matcher, $\mathcal{E}(E)$ denote the set of matches produced by $\mathcal{E}$ on entities E . Let $M \in \{\text{SMP}, \text{MMP}\}$ be a message passing technique and let $M(E)$ denote the set of matches produced by our framework using M . Note that the absolute precision/recall numbers of M are not the right metric to evaluate our framework, since these numbers are tied to the underlying entity matcher $\mathcal{E}$. Instead, we define the following four properties.

1. *Soundness* : this is the fraction of matches in $M(E)$ that are also in $\mathcal{E}(E)$. We say that M is *sound* if it has

soundness 1, i.e., $M(E) \subseteq \mathcal{E}(E)$.

2. *Completeness* : this is the fraction of matches in $\mathcal{E}(E)$ that are also in $M(E)$. we say that M is *complete* if it has completeness 1, i.e., $\mathcal{E}(E) \subseteq M(E)$.
3. *Consistency* : we say that M is consistent if the final set of matches do not depend on the order in which covers are evaluated and messages are exchanged.
4. *Scalability* : we want the time complexity of M to be low, preferably linear in the number of covers. Here, we assume that the sizes of covers are bounded, and study scalability with respect to the number of covers.

In Sec. 4, we define certain natural, intuitive properties on entity matchers and say that a matcher is *well-behaved* if it satisfies this properties. Our main result is as follows.

THEOREM 1. *If $\mathcal{E}$ is a well-behaved entity matcher, then both SMP and MMP are sound, consistent, and have time complexity linear in the number of covers.*

While there are no theoretical guarantees on completeness, we demonstrate empirically that our framework is in fact complete on the data sets we consider. In the following sections, we formally define entity matchers, the notion of *well-behaved* matcher, various message passing schemes, and revisit Theorem 1.

4. BLACK-BOX ABSTRACTION

We define two blackbox abstractions for an entity matcher. The first, which we call a *Type-I matcher*, is a deterministic matcher that outputs a set of matches. This is the most general abstraction that models any entity matching algorithm. The second abstraction, which we call a *Type-II matcher*, gives a probability distribution on sets of possible matches. As we shall see, a Type-II matcher is a special kind of Type-I matcher. However, we introduce the second abstraction because we can apply additional techniques in our framework when the matcher is probabilistic. We describe below the two abstractions.

4.1 Type-I Matcher

Recall from Section 2 that an instance of entity matching problem consists of a set of entities E along with a set of attributes for each entity and a set of relationships $\mathcal{R}$ defined on them. In the rest of the paper, when will use E to denote a set of entities and we will implicitly assume the presence of attributes and relationships on entities.

DEFINITION 1 (TYPE-I MATCHER). *A Type-I matcher is a function $\mathcal{E} : E \times 2^{(E \times E)} \times 2^{(E \times E)} \mapsto 2^{(E \times E)}$.*

The first argument to $\mathcal{E}$ is a set of entities, the next two arguments are sets $V_+ \subseteq (E \times E)$ and $V_- \subseteq (E \times E)$, which are called the evidence sets. The set V_+ is a set of entity pairs that are known to be matches, while the set V_- is a set of entity pairs that are known to be non-matches. The output of the function $O \subseteq E \times E$ is the set of pairs of entities declared as matches by the algorithm.

Note that this definition treats the entity matching module as a complete black box except for a mechanism to provide evidence, which we need in our framework.

All the matchers we discussed in Section 2 can be modeled as a Type-I matcher. For non-collective matchers, the evidence sets do not serve any purpose, since all match decisions are independent of each other. However, for collective entity matchers, match decisions are interrelated. So the evidence sets affect their output matches. All the collective matchers we discussed in Section 2 can incorporate a mechanism to accept evidence sets V_+ and V_- . The iterative matchers [4, 7] can incorporate V_+ by starting their iterations from the set V_+ instead of the empty set. However, in this matchers, the set V_- does not play any role. The purely collective matchers based on probabilistic models can incorporate the evidence by conditioning on V_+ and V_- , i.e., setting the matches in V_+ to **true** and V_- to **false** during probabilistic inference.

We assume that the following two axiomatic properties for a Type-I matcher. These properties are natural and we expect all entity matchers to satisfy them.

DEFINITION 2. *A Type-I matcher satisfies the following properties for all E, V_+ and V_- .*

(i) **Idempotence:** *If $O = \mathcal{E}(E, V_+, V_-)$, then*

$$\mathcal{E}(E, O, V_-) = O$$

(ii) **Closure:** *for all E' such that $E' \cap E = \emptyset$,*

$$\mathcal{E}(E, V_+, V_-) = \mathcal{E}(E \cup E', V_+, V_- \cup (E' \times E))$$

The idempotence property implies that the output of the matcher will not change if the output itself is again given as the positive evidence. Closure implies that the entity match lies in a *closed world*, so we can obtain the output for E by simply running $\mathcal{E}$ on $E \cup E'$ with the additional negative evidence that all pairs involving entities in E' are non-matches.

In addition to the two properties, we state a third property of entity matchers that allow us to efficiently implement them in our framework.

DEFINITION 3. (Monotonicity) *A Type-I entity matcher is monotone if for any entity set E and evidence sets V_+, V_- the following holds: (i) for any set $V_+' \supseteq V_+$, $\mathcal{E}(E, V_+') \supseteq \mathcal{E}(E, V_+)$, and (ii) for any $V_-' \supseteq V_-$, $\mathcal{E}(E, V_-') \subseteq \mathcal{E}(E, V_-)$.*

Monotonicity means that if we give more positive evidence, we get more matches in the output, while if we give more negative evidence, we get less matches in the output.

DEFINITION 4. *We say that a matcher is well-behaved if it satisfies idempotence, closure and monotonicity.*

4.2 Type-II Matcher

DEFINITION 5 (TYPE-II MATCHER). *A Type-II matcher $\mathcal{E}$ takes as input a set of entities E and associates a probability distribution $P_{\mathcal{E}}$ on the set $2^{E \times E}$.*

The probability of a set $S \subseteq E \times E$, $P_{\mathcal{E}}(S)$, is the probability that exactly the pairs in S are matches.

Note that we have not specified an explicit mechanism to specify the initial evidence sets. This is because, since the matcher gives a complete probability distribution, any evidence sets V_+, V_- can easily be incorporated by conditioning the distribution $P_{\mathcal{E}}$ on matches in V_+ to be **true** and V_- to be **false**.

The output of a type-II matcher $\mathcal{E}$ is the set $S \in 2^{E \times E}$ that has the highest probability according to the distribution $P_{\mathcal{E}}$. We denote it by $\mathcal{E}(E)$. There can be more than one most likely set, in which case, we will prefer a set with the largest size. More precisely, $\mathcal{E}(E)$ is any set $S \subseteq 2^{E \times E}$ that satisfies the following two properties: (i) for all S' , $P_{\mathcal{E}}(S) \geq P_{\mathcal{E}}(S')$ and (ii) for all S' such that $P_{\mathcal{E}}(S) = P_{\mathcal{E}}(S')$, we have $|S| \geq |S'|$.

In presence of evidence sets V_+ and V_- , we can incorporate it in the entity matcher by simply conditioning the probability distribution $P_{\mathcal{E}}$ on V_+ and V_- . Thus, we define $\mathcal{E}(E, V_+, V_-)$ to denote any set among the largest most likely sets that contains all matches in V_+ , but none from V_- .

PROPOSITION 1. *A Type-II entity matcher is also a Type-I matcher, i.e. it satisfies the idempotence and closure axioms.*

We also define a supermodularity property on Type-II matchers that is related to the monotonicity property of Type-I matchers.

DEFINITION 6 (SUPERMODULARITY). *An entity matcher $\mathcal{E}$ is supermodular if for all entity pair sets $S, T \subseteq E \times E$ such that $S \subseteq T$, and for all entity pairs $p \in E \times E$, we have*

$$\frac{P_{\mathcal{E}}(T \cup \{p\})}{P_{\mathcal{E}}(T)} \geq \frac{P_{\mathcal{E}}(S \cup \{p\})}{P_{\mathcal{E}}(S)}$$

Intuitively, supermodularity says that the matches are positively correlated: increasing the set of matches from S to a larger set T only increases the likelihood of another match p . Most entity matching algorithms in the literature infer only new matches from other matches, and hence are supermodular.

Supermodularity is a well-studied property of probability distributions (see [13]). In fact, exact inference techniques are known for Markov Networks when the clique potentials are supermodular [22, 13]. However, these work only if the cliques are edges (while we support any general entity matcher), and have super-linear complexity (quadratic or worse). Thus such techniques are not well-suited for extremely large data sets that we are interested in.

We also show that supermodularity and monotonicity are related. The following proposition is proved in the full version of the paper [10] explains the relationship.

PROPOSITION 2. *Let $\mathcal{E}$ be a supermodular Type-II entity matcher. Then $\mathcal{E}$ is a monotone Type-I entity matcher.*

5. COVERING

In this section, we define the notion of a covering, and introduce a class of coverings called *total covering*, which are especially suited for our framework. Given a set of entities E , a *covering* of E is simply a set $\mathcal{C} = \{C_1, C_2, \dots, C_k\}$ such that $E = C_1 \cup \dots \cup C_k$. Each C_i is called a *cover*.

EXAMPLE 2. *Consider the set of entities shown in Figure 1 with relations $\mathcal{R} = \{\text{Coauthor}, \text{Similar}\}$ over E as defined in Sec. 3.1. Let C_1, C_2 and C_3 be the sets as shown in Figure 2. Then, $C_1 = \{C_1, C_2, C_3\}$ is a covering. The set $C_2 = \{C_1, C_3\}$ is also a covering.*

Given a relation R over E , and a subset $C \subseteq E$, let $R(C)$ denote the subset of the relation induced by C , i.e. the set

of tuples in R that solely consists of entities from C . Thus, $R(E)$ is the complete relation.

Next we define the notion of total covering that extends the notion of a cover.

DEFINITION 7 (TOTAL COVERING). *Let E be a set of entities along with a set of relations $\mathcal{R}$. A covering $\mathcal{C} = C_1, C_2, \dots, C_k$ is a total covering w.r.t $\mathcal{R}$ if for all relations $R \in \mathcal{R}$, we have $R(E) = R(C_1) \cup \dots \cup R(C_k)$*

EXAMPLE 3. *In Example 2, $\mathcal{C}_2$ is not a total covering, since the tuple $R(b_1, c_1)$ is not contained in either of the two covers. However, one can verify that the covering $\mathcal{C}_1$ is a total covering.*

Total covering is a desirable property in our framework, since tuples in R that are not included in any cover get “lost”, as they do not participate at all in the matching process. Given any covering, we can make it a total covering as follows: Define the boundary of a cover C to the set of entities e for which there is an entity e' in C such that both e and e' occur together in some tuple in $\mathcal{R}$. Then, by expanding each cover to also include its boundary, we obtain a total cover.

Total covering is a natural extension of the notion of *blocking*, used extensively in entity matching [20, 15]. Blocking divides entities into sets such that for any pair of similar entities, there is a set that contains the pair. Thus, blocking is a total covering over the *Similar* relation. When we have other relations, like *Coauthor* and *Cites*, we can have covers containing entities that are not similar, and even entities that are not of the same type, e.g an author and a paper.

In this work, we construct a total covering by first constructing a total covering over *Similar* relation using the Canopies algorithm [15], and then taking the boundary of each cover with respect to other relations.

6. MESSAGE PASSING

Finally, we describe the message passing algorithm, which is at the heart of our framework. The input is a covering $\mathcal{C} = C_1, C_2, \dots, C_k$ of entities E and an entity matcher $\mathcal{E}$, which is assumed to be *well-behaved*.

Recall that $\mathcal{E}$ takes a set of entities C , a set of positive evidence V_+ and a set of negative evidence V_- . We use $\mathcal{E}(C)$ to denote the output of the matcher when there is no evidence, i.e. when $V_+ = V_- = \emptyset$. Also, we use $\mathcal{E}(C_i, V_+)$ to denote the output when $V_- = \emptyset$.

We describe two message passing schemes. The first, called the simple message passing scheme, works for any matcher. The second, which is the maximal message passing scheme, is more advanced but only works with Type-II matchers, i.e. matchers which are probabilistic.

6.1 Simple Message Passing Scheme (SMP)

6.1.1 The Algorithm

We described the algorithm informally in Sec. 3.2. The algorithm maintains a set A of active covers, which are the covers that can potentially produce new matches. A is initialized to the set of all covers. As $\mathcal{E}$ is run on a cover C , it is removed from A . However, all the covers that are affected by the new matches discovered from C are put back into the set of active covers. The algorithm terminates when A becomes empty. Algorithm 1 below describes this procedure.

For a set of entity pairs S , the function $Neighbour(S)$ denotes the set of all covers C such that for some $(e_i, e_j) \in S$, we have both $e_i, e_j \in C$.

Algorithm 1 SMP **Inputs:** Entity Matcher $\mathcal{E}$, A covering $\{C_1, \dots, C_s\}$

```

1:  $A \leftarrow \{C_1, \dots, C_s\}$  // set of active covers
2:  $M_+ \leftarrow \emptyset$  // set of matches found
3: while  $A \neq \emptyset$  do
4:    $C \leftarrow$  remove a cover from  $A$ 
5:    $M_C = \mathcal{E}(C, M_+)$ 
6:    $A = A \cup Neighbour(M_C - M_+)$ 
7:    $M_+ = M_+ \cup M_C$ 
8: end while
9: return  $M_+$  as the set of final match.
```

EXAMPLE 4. *We illustrate how Algorithm 1 works by applying the MLN matcher of Sec. 3.1 on the covering given in Figure 2. Initially, the set of active covers is $A = \{C_1, C_2, C_3\}$. In the first step, C_1 is removed from A and $\mathcal{E}$ is run over it. As we explain in Sec. 3.1, this does not result in any match. Next, C_2 is removed from A and processed. Again this does not result in any match. Finally C_3 is removed and processed. This produces the match $Match(c_1, c_2)$. Since both c_1 and c_2 are contained in cover C_2 , C_2 is put back into A . Thus, it is processed again in the next step and this time it produces $Match(b_1, b_2)$, since it now has the evidence of a match between c_1 and c_2 . Since b_1 and b_2 are both in C_1 , it is pushed back into A and processed. This does not result in any new match. At this point, A is empty and algorithm terminates, producing the two matches $Match(c_1, c_2)$ and $Match(b_1, b_2)$.*

6.1.2 Analysis of SMP

We revisit here the Theorem 1 for SMP, and show that SMP is sound, consistent and has nice scaling properties.

THEOREM 2. *Let $\mathcal{E}$ be any well-behaved Type-I matcher. Then the following hold for SMP:*

1. (Convergence) SMP terminates after finite steps.
2. (Soundness) The set of matches produced by SMP is a subset of $\mathcal{E}(E)$.
3. (Consistency) The output of SMP is invariant under the order in which covers are evaluated.

Proof: Part (1) follows easily, since every time a cover is added to the set A of active covers, the set M strictly increases. Since the size of M is bounded by $|E \times E|$, the algorithm must terminate.

To show (2), we need more results. We first define the notion of a sound message. Given a set $M \in E \times E$, we say that M is *sound* if $M \subseteq \mathcal{E}(E)$, i.e. M represents correct set of matches with respect to the matcher. Next we state the following result.

PROPOSITION 3. *Let $\mathcal{E}$ be a well-behaved matcher, C be any cover, and M^+ be a sound set of matches. Then (a) $\mathcal{E}(C, M_+) \subseteq \mathcal{E}(E, M_+)$, and (b) $M_C = \mathcal{E}(C, M_+)$ is sound.*

Proof of Prop. 3: We prove (a) first. Denote $V_- = (E - C) \times E$. As $\mathcal{E}$ is consistent we know that $\mathcal{E}(C, M_+) = \mathcal{E}(E, M_+, V_-)$. As $\mathcal{E}$ is monotone,

$$\mathcal{E}(E, M_+, V_-) \subseteq \mathcal{E}(E, M_+, \emptyset) = \mathcal{E}(E, M_+)$$

Combining, we get the result that $\mathcal{E}(C, M_+) \subseteq \mathcal{E}(E, M_+)$.

We prove (b) using contradiction. We know from (a), $\mathcal{E}(C, M_+) \subseteq \mathcal{E}(E, M_+)$. As M_+ is sound, we know that $M_+ \subseteq \mathcal{E}(E)$. Thus, using monotonicity, we get

$$\mathcal{E}(E, M_+) \subseteq \mathcal{E}(E, \mathcal{E}(E)) = \mathcal{E}(E)$$

where the last equality follows from the closure property. Combining, we get $M_C = \mathcal{E}(C, M_+) \subseteq \mathcal{E}(E, M_+) \subseteq \mathcal{E}(E)$ showing that M_C is sound. $\square$

Finally, using Proposition 3, we prove part (2) of Theorem 2. We use an induction argument. Assume that M_+ is sound at the beginning of some iteration (base case is true as $M_+ = \emptyset$ for the first iteration). The above Proposition guarantees that the $M_C = \mathcal{E}(C, M_+)$ is sound. Thus the new M_+ , which is the union of M_+ and M_C is again sound proving the induction step and completing the proof of part (2).

The proof of part (3) is omitted and appears in an extended version of this paper.

Running Time Analysis Let n be the number of covers, k be the maximum size of each cover, and let $f(k)$ be the running time of $\mathcal{E}$ on a cover of size k . For a cover C , the maximum set of matches it can produce is $C \times C$, which has cardinality at most k^2 . Each time C is added to the set A , the set $(C \times C) - M_+$ strictly decreases. Thus C can be added to A at most k^2 times. Hence, we get the following result:

THEOREM 3. *The time complexity of SMP is $O(k^2 f(k)n)$, where n is the number of covers, k is the maximum size of a cover and $f(k)$ is the running time of $\mathcal{E}$.*

Assuming that the size of covers is bounded, the time complexity is linear in the number of covers (and hence, linear in the number of entities). In practice, a cover is never evaluated k^2 times, and the running time is much lower than the theoretical upper bound.

6.2 Maximal Message Passing Scheme (MMP)

6.2.1 Motivation

In Sec. 3.2, we gave an example demonstrating the weakness of SMP. Let us revisit this example. Consider the three pairs (a_1, a_2) , (b_2, b_3) and (c_2, c_3) in Figure 2. We showed in Sec. 3.1 that matching all the three pairs increases the overall score if and only if all the three of them are matched. Thus we have a ‘chicken and egg’ problem : (i) *MLN* can match (a_1, a_2) and (b_2, b_3) when run on C_1 if and only if it is given (c_2, c_3) as evidence, and (ii) *MLN* can match (b_2, b_3) and (c_2, c_3) when run on C_2 if and only if it is given the match (a_1, a_2) . For SMP to discover the three matches, we need both (i) to happen before (ii), and (ii) to happen before (i). Consequently, none of the three matches are output by the SMP algorithm. The maximal message passing scheme described below overcomes this problem.

6.2.2 The Algorithm

The maximal message passing scheme (MMP) requires a probabilistic entity matcher. So we assume that $\mathcal{E}$ is associated with a probability distribution $P_{\mathcal{E}}$ that gives the probability of any given set of matches.

We first define below the notion of a maximal message.

DEFINITION 8 (MAXIMAL MESSAGE). *A maximal message w.r.t matcher $\mathcal{E}$ is a set of pairs $M \subseteq E \times E$ such that $M \subseteq \mathcal{E}(E)$ or $M \cap \mathcal{E}(E) = \emptyset$.*

In other words, a maximal message is a set of pairs such that either all of them will be matched by $\mathcal{E}$ or none of them will be matched. The following proposition follows easily from the definition of a maximal message.

PROPOSITION 4. (i) *If M is a maximal message then so is any $M' \subseteq M$, and (ii) If M and M' are two overlapping maximal messages (i.e. $M \cap M' \neq \emptyset$), then $M \cup M'$ is also a maximal message.*

Next we describe an algorithm that computes a set of maximal messages for a cover C . It takes as input a set of sound evidence matches M_+ .

Algorithm 2 COMPUTEMAXIMAL (**Inputs:** Entity Matcher $\mathcal{E}$, Cover C , Evidence M_+)

- 1: For each pair $p = (e_1, e_2)$ of entities in C , compute $\mathcal{E}(C, M_+ \cup \{p\})$
 - 2: Construct a graph $G(V, E)$ defined as follows:
 - V contains a node corresponding to each pair p of entity references in C .
 - E contains an edge between pairs p and p' if $p \in \mathcal{E}(C, M_+ \cup \{p'\})$ and $p' \in \mathcal{E}(C, M_+ \cup \{p\})$.
 - 3: For each connected component of G , output a message consisting of all the pairs in the component.
-

We defer the proof of correctness of COMPUTEMAXIMAL to Sec. 6.2.3, and instead illustrate the algorithm here on an example.

Consider cover C_2 in Figure 2. It contains four pairs (b_1, b_2) , (c_1, c_2) , (b_2, b_3) and (c_2, c_3) ². Assume that M_+ is empty to start with. If we compute $\mathcal{E}(C_2, M_+ \cup \{b_2, b_3\})$, we will get (c_2, c_3) as a match since matching it only incurs a weight of -5 by rule R_1 while it gets a weight of +8 by rule R_2 . Similarly, $\mathcal{E}(C_2, M_+ \cup \{c_2, c_3\})$ results in a match of (b_2, b_3) . Thus, we add an edge between the two pairs in Step 2 of the algorithm. In the same manner, we also obtain an edge between (b_1, b_2) and (c_1, c_2) . The graph G has two connected components, corresponding to the two messages $\{(b_1, b_2), (c_1, c_2)\}$ and $\{(b_2, b_3), (c_2, c_3)\}$.

The main principle behind MMP algorithm is to exchange maximal messages between covers. The idea is that two maximal messages can combine to produce a sound message, if they together increase the probability of the solution. In our example, cover C_2 produces a maximal message $\{(b_2, b_3), (c_2, c_3)\}$ and cover C_1 produces a maximal message $\{(a_1, a_2), (b_2, b_3)\}$. When these two messages are combined, the new maximal message has enough evidence to match all its three pairs, making it a sound message.

²Technically, the cover also contains pairs like (b_1, c_2) but we can restrict ourselves to pairs which are similar, since $\mathcal{E}$ never considers pairs that are not similar.

We now formally define the MMP algorithm (Algorithm 3). The algorithm, similar to SMP, maintains a set of active covers. It maintains a set M_+ of matches found so far. In addition, it maintains a set T of maximal messages found so far. In Step 5, when a cover is processed, we find both the new matches M_C and new maximal messages T_C . In Step 6, we use Proposition 4 to combine maximal messages from two covers. Proposition 4 says that a set of maximal messages can be equivalently represented by replacing overlapping messages with their unions. Given a set of (possibly overlapping) maximal messages T , let T^* denote the set obtained from T by repeatedly picking two messages from T that overlap and replacing them with their union, until all the messages are disjoint. Thus, we update the set T by taking its union with the new maximal messages T_C and then taking $(T \cup T_C)^*$. Step 7 checks if a maximal message can be converted to a sound message.

Step 7 is the only step that requires $\mathcal{E}$ to be a probabilistic matcher, as it involves a check $P_{\mathcal{E}}(M_+ \cup M) \geq P_{\mathcal{E}}(M_+)$.

Algorithm 3 Mmp **Inputs:** Entity Matcher $\mathcal{E}$, covering $C = \{C_1, \dots, C_k\}$

```

1:  $A \leftarrow \{C_1, \dots, C_s\}$  // set of active covers
2:  $M_+ \leftarrow \emptyset$  // set of matches found
    $T \leftarrow \emptyset$  // set of maximal messages found
3: while  $A \neq \emptyset$  do
4:    $C \leftarrow \text{remove a cover from } A$ 
5:    $M_C = \mathcal{E}(C, M_+)$ 
      $T_C = \text{ComputeMaximal}(C, M_+)$ 
6:    $M_+ = M_+ \cup M_C$ 
      $T = (T \cup T_C)^*$ 
7:   Find sound maximal messages: Check if there
     exists  $M \in T$  s.t.  $P_{\mathcal{E}}(M_+ \cup M) \geq P_{\mathcal{E}}(M_+)$ . If yes,
     then update  $M_+ = M_+ \cup M$ .
8:    $A = A \cup \text{Neighbour}(M_C \cup M - M_+)$ 
9: end while
10: return  $M_+$  as the set of final match.
```

Algorithm 3 assumes that we can efficiently compute $P_{\mathcal{E}}$ for any set of inputs. This condition, in general, is true for probabilistic models, where finding the optimal solution, i.e. $\arg \max_S P_{\mathcal{E}}(S)$ is a very expensive operation that involves searching over a large space, but computing $P_{\mathcal{E}}(S)$ for a specific S is very cheap using the parameters of the model.

Finally, to finish our example, we see that the maximal message from covers C_1 and C_2 get combined to produce a maximal message $\{(a_1, a_2), (b_2, b_3), (c_2, c_3)\}$ which satisfies the condition of Step 7, since the total weight of matching all the three pairs is more than the total weight of matching none of them, and hence all the three pairs get matched. We skip the details of how the rule weights translated into probabilities by the MLN.

6.2.3 Analysis of MMP

We first give the proof of correctness of the algorithms. First we show that the COMPUTEMAXIMAL algorithm correctly outputs maximal messages.

LEMMA 1. *Let $\mathcal{E}$ be any well-behaved type-I matcher, M_+ be any sound evidence set, and C be any cover. Then all the messages produced by COMPUTEMAXIMAL are maximal.*

PROOF. Let M be any message in the output corresponding to a connected component CC of the graph G . We shall

show that M is maximal. For this we use induction.

Base case: The set $M = \{p, p'\}$ is maximal if there is an edge between p and p' in G . To prove this recall the definition of maximality. For showing maximality of a two element set $M = \{p, p'\}$, w.l.o.g, it is sufficient to show that: $p \in \mathcal{E}(E, M_+)$ implies $p' \in \mathcal{E}(E, M_+)$. Assume the contradiction, i.e. $p \in \mathcal{E}(E, M_+)$ but $p' \notin \mathcal{E}(E, M_+)$. Since $p \in \mathcal{E}(E, M_+)$, the set $\{p\}$ is sound. Moreover M_+ is also sound implying that the set $M_+ \cup \{p\}$ is sound. Since p and p' are connected by an edge in G , we know that $p' \in \mathcal{E}(C, M_+ \cup \{p\})$. Applying Proposition 3(b) for the sound message $M_+ \cup \{p\}$ and cover C , we get that p' is also sound. This means that $p' \in \mathcal{E}(E, M_+)$, which contradicts our assumption that $p' \notin \mathcal{E}(E, M_+)$. Thus, the set $M_+ = \{p, p'\}$ is maximal.

Inductive hypothesis: Assume M is maximal for all connected components CC having at most k nodes.

Inductive Step: Decompose any connected component CC of $k+1$ nodes into two connected components of CC_1 and CC_2 , such that CC_1 and CC_2 share a node and have at most k nodes each. This is always possible for any connected component CC . By inductive hypothesis M_{CC_1} and M_{CC_2} are both maximal. Moreover since CC_1 and CC_2 share a node, the messages $M_{CC_1} = \cup_{p \in CC_1} \{p\}$ and $M_{CC_2} = \cup_{p \in CC_2} \{p\}$ have non-empty intersection. Applying Proposition 4, we obtain that $M = M_{CC_1} \cup M_{CC_2}$ is also maximal. Hence proved. $\square$

THEOREM 4. *Let $\mathcal{E}$ be a supermodular Type-II matcher. Then, MMP always terminates. Moreover on termination, the output M_+ is sound (i.e. $M_+ \subseteq \mathcal{E}(E)$) and consistent (i.e. invariant under the order in which covers are processed)*

PROOF. (Informal) The proof is similar to the inductive proof of soundness of for the SMP Algorithm. In MMP, there is just one extra step (Step 7) that affects M_+ . In Step 7, we add the pairs in M to the existing sound message M_+ . All we need to show is that M is sound. Then we ensure that the update done in Step 6 preserves the soundness of M_+ , and hence the theorem will follow in a way similar to the proof for SMP.

To show M is sound, note that $P_{\mathcal{E}}(M_+ \cup M) \geq P_{\mathcal{E}}(M_+)$. Let $O = \mathcal{E}(E)$. By supermodularity of $\mathcal{E}$, we know that $\frac{P_{\mathcal{E}}(O \cup M)}{P_{\mathcal{E}}(O)} \geq \frac{P_{\mathcal{E}}(M_+ \cup M)}{P_{\mathcal{E}}(M_+)} \geq 1$. Thus $P_{\mathcal{E}}(O \cup M) \geq P_{\mathcal{E}}(O)$, which is only possible if $M \subseteq O$. Hence M is sound. This completes the proof. $\square$

Running Time Analysis Let n denote the number of covers, k be the maximum size of each cover, and let $f(k)$ be the running time of $\mathcal{E}$ on a cover of size k . As in the case of SMP, each cover can be processed at most k^2 times. However, unlike SMP, processing a cover is more expensive for MMP. It involves two things: a call to COMPUTEMAXIMAL and a computation of $(T \cup T_C)^*$.

COMPUTEMAXIMAL calls $\mathcal{E}$ on each pair in the cover, which potentially results in k^2 calls to the matcher. Thus, COMPUTEMAXIMAL runs in time $k^2 f(k)$. To bound the time for computing $(T \cup T_C)^*$, observe that we can use the disjoint-set data structure to maintain connected components in T and use the union-find algorithm to update it. Further, the total size of the set T_C can be at most k^2 (since each pair in the cover can be in at most one connected component). Thus, $(T \cup T_C)^*$ computation involves at most k^2 union-find operations. Each operation is known to have a constant amortized

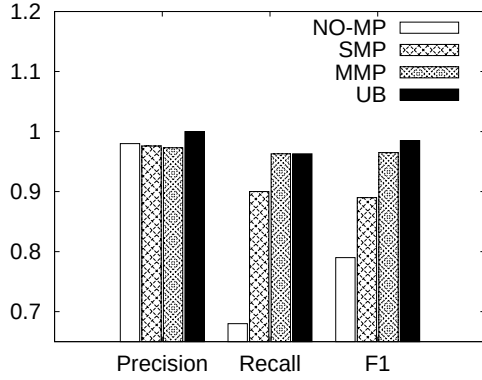


Figure 3: Precision/Recall/F1 scores on HEPHT

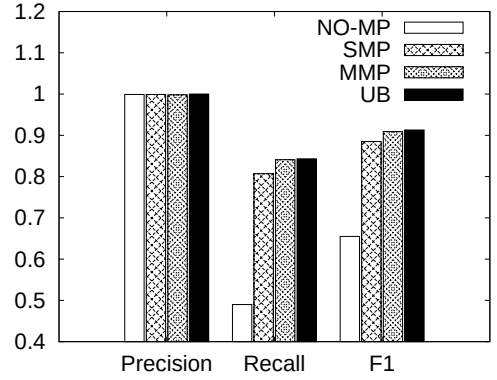


Figure 4: Precision/Recall/F1 scores on DBLP

cost³. Total cost of updating T can thus be bounded by $O(k^2)$ and hence the total cost of processing a cover can be bounded by $k^2 f(k) + O(k^2) = O(k^2 f(k))$.

Hence, we get the following result:

THEOREM 5. *MMP runs in time $O(k^4 f(k)n)$, where n is the number of covers, k is the maximum size of a cover and $f(k)$ is the running time of $\mathcal{E}$.*

Similar to SMP, the time complexity is linear in the number of covers. Again, in practice, a cover is never evaluated k^2 times, and the running time is much lower than the theoretical upper bound.

7. EXPERIMENTS

We evaluate the performance, both accuracy and efficiency, of our message passing schemes. For evaluation, we use the collective entity matching algorithm of Singla et al. [21], which is a state of the art algorithm, and uses Markov Logic Networks. For covering, we use the *Canopy* algorithm given by McCallum et al. [15]. Our message passing algorithms are written in Java. All our experiments are run on an Intel Xeon® 2.13GHz machine running linux with 4GB RAM.

Datasets We use two datasets. The HEPHT dataset is a dataset used in the 2003 KDD Cup and available for download from the KDD website. It contains information on papers in theoretical high-energy physics. It has 58,515 author references mentioned across 29,555 papers. For collective entity matching, we only use the names of authors and the co-authorship relation on them. The HEPHT dataset has a completely labeled ground truth, which give for each author reference, the author it refers to. The 58,515 author references resolve into 13092 distinct authors using this labeled ground truth.

Since no other large labeled dataset is publicly available, we manually prepared a second dataset, DBLP, as follows. From the DBLP Computer Science Bibliography [14], we obtained publications corresponding to a large number of Database and AI conferences. We compiled 19,408 papers with a total of 50195 author references, with 21,278 distinct

authors. However, since DBLP data is clean, we manually add noise by randomly adding small mutations to author names. Thus, the original names serve as the ground truth in evaluating this dataset.

7.1 Accuracy of message passing schemes

Precision/Recall In the HEPHT dataset, the covering algorithm results in 13K covers containing a total of 1.3M entity pairs. We compare three algorithm: NO-MP is the no message passing scheme, where EM is run on each cover but no messages are exchanged. SMP is the simple message passing scheme. MMP is the maximal message passing scheme. For each scheme, we compute the precision, recall and the $F1$ score, which is the harmonic mean of their precision and recall.

We also want to compare these numbers with those for running running the EM on the whole dataset. However, running EM at this scale is infeasible. For instance, on HEPHT, it involves creating an Markov Logic Network with 1.3M nodes and running inference over it. Instead, we use the following technique to obtain an upper bound on the set of matches that EM can produce. For each entity pair, we give the EM algorithm the ground truth about all other entity pairs and run the matcher to decide the given entity pair. Since our matcher satisfies the supermodularity property, we can show that this is indeed an upper bound on the set of matches that EM can produce. We call this scheme UB. Note that this is not an algorithm since it uses the ground truth.

To compute an upper bound on $F1$, we compute the recall of UB and assume a maximum precision of 1, since precision of UB might be lower than running EM on the whole data.

Figure 3 plots these scores for various schemes on HEPHT. In each of the message passing schemes, the precision is very high and close to 1. This is because all the message passing schemes are sound, and the original EM algorithm itself has a high precision. Also, MMP comes close to UB. Note that the UB need not be attainable by any message passing scheme, since the original EM itself may have a recall less than that of UB and precision less than 1.

Figure 4 shows the same graph for DBLP. The DBLP dataset produces 30K covers containing a total of 0.5M entity pairs. Note that while both the datasets have roughly the same number of author references, DBLP produces more

³Technically, the cost is given by the inverse Ackerman function $\alpha(n)$ which, for all practical purposes, can be assumed to be a small constant.

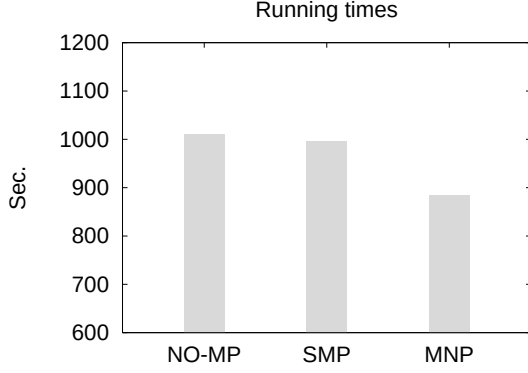


Figure 5: Running Times Comparison on HEPH

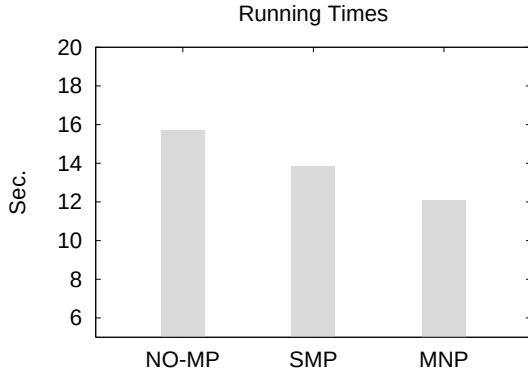


Figure 6: Running Times Comparison on DBLP

than twice the number of covers as HEPH, with a much smaller average cover size (as evident by the less number of entity pairs). This is due to the fact that DBLP always contains the full author names as opposed to HEPH, where names are often abbreviated, leading to more name clashes and fewer covers of larger size.

Soundness/Completeness

We know that our message passing schemes are sound, but not necessarily complete. Hence, we study the completeness of various algorithms empirically. Recall that completeness is the fraction of the matches found by the EM algorithm that are retrieved by a particular scheme. Again, since we do not have the true set of matches given by EM on the whole dataset, we use the matches of UB to obtain an upper bound on completeness.

Figure 7 shows the completeness for various schemes with respect to UB. We see that MMP has completeness 1 for HEPH. Thus, MMP is both sound and complete and hence, gives the exact same output as running the EM on the whole HEPH dataset. Similarly, for DBLP, MMP has completeness nearly 1.

7.2 Running Times

Figure 5 shows the running times of the various message passing schemes on HEPH. We observe an interesting and counter-intuitive behavior. SMP, which does the extra work

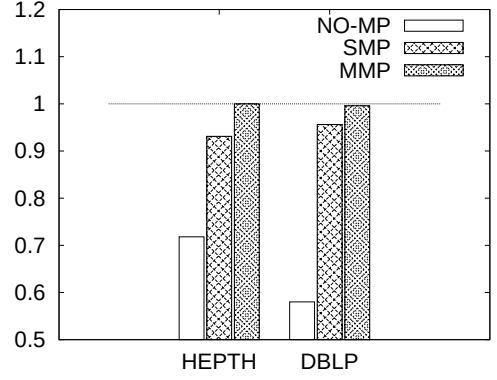


Figure 7: Completeness of Message Passing Schemes

of passing messages as well as running covers multiple times, has a lower running time than NO-MP, that does not do message passing at all. MMP, that passes more messages, has an even lower running time. This apparent paradox can be explained as follows. In all the three schemes, the total running time is dominated by the sum of running times of EM on all the covers. The actual overhead of message passing is minimal. However, MMP wins over NP-MP because, since covers share entities, messages often reduce the *active size* of the covers. All the pairs in a cover that have already been matched by other covers are not part of inference anymore. Hence, EM runs faster. For instance, if a cover has 100 entity pairs to consider, and each pair has already been matched by some previous cover, EM does not need to do any work on this cover. The net result is that even though SMP has to revisit covers multiple times, the *active sizes* of covers keep getting lower, and the total running time is lower. MMP, by a more effective message passing, achieves an even lower running time by the same principle. Thus a better message passing scheme not only increases the precision/recall of the matching, it also results in a lower running time.

Figure 6 shows the running times on DBLP dataset. DBLP exhibits a similar behavior. However, we observe an interesting difference. While both the datasets have roughly the same size, the running times on DBLP are orders of magnitude lower than HEPH. This is because the covers in DBLP are much smaller, owing to reasons explained earlier. As a result, EM runs significantly faster on the covers in DBLP.

Cover scheduling The order in which covers are processed has a significant effect on the overall running time. Since larger covers run significantly slower than smaller covers, we want to process smaller covers first, so that when larger covers are processed, their active size as discussed above is potentially smaller and EM runs faster on them. All the numbers reported for SMP and MMP are based on scheduling shortest covers first. To study the impact of this optimization, we create a variant of SMP and MMP in which covers are scheduled in the reverse order of their sizes. We call this SMP^R and MMP^R . We find that on HEPH dataset, SMP^R takes 5008 seconds while MMP^R takes 1374 seconds. Thus, ordering does have a significant impact on running time with a blowup of about 5 times for the SMP algorithm. The blowup in time is owing to the

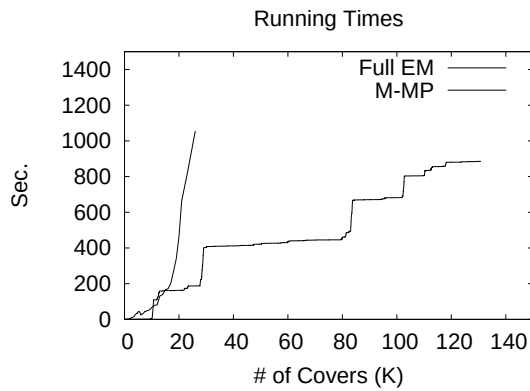


Figure 8: Running Times on HEPH

fact that several large covers have to be revisited when new matches are subsequently found in smaller covers.

Finally, we analyze the running time of EM on the whole dataset. Figure 8 shows, for each k , the total running time of EM when run on the first k covers together. The graph shows an exponential behavior of EM, and it becomes prohibitively expensive to run EM on more than 2500 covers. On the other hand, our message passing scheme, MMP, exhibits a linear behavior for the whole 13,000 covers. At certain points, we see jumps in the curve of MMP. This is because some of the covers are large, and whenever a new large cover is included, the running time shows a small jump.

8. CONCLUSION AND FUTURE WORK

We described a framework to scale collective entity matching algorithms. The framework is generic and can incorporate any matcher as a blackbox. We provide various message passing schemes, and show that for ‘well-behaved’ matchers, our framework is guaranteed to be sound. In experiments over real-life large data sets, we also show that the framework also achieves near-completeness.

As a future work, we want to understand the full-scope of this message passing framework. We want to characterize the class of matchers where message passing schemes guarantee completeness. We also want to extend the notion of messages to include probabilistic information to try to provide completeness. Also, currently we assume that matchers are supermodular. We want to extend our framework and devise message passing schemes to handle arbitrary entity matchers.

9. REFERENCES

- [1] Rohit Ananthakrishna, Surajit Chaudhuri, and Venkatesh Ganti. Eliminating fuzzy duplicates in data warehouses. In *VLDB*, pages 586–596, 2002.
- [2] Brian Babcock and Surajit Chaudhuri. Towards a robust query optimizer: a principled and practical approach. In *SIGMOD*, pages 119–130, 2005.
- [3] Indrajit Bhattacharya and Lise Getoor. A latent dirichlet model for unsupervised entity resolution. In *SIAM Conference on Data Mining (SDM)*, 2006.
- [4] Indrajit Bhattacharya and Lise Getoor. Collective entity resolution in relational data. *ACM Trans.*

- Knowl. Discov. Data*, 1(1), 2007.
- [5] William W. Cohen, Pradeep Ravikumar, and Stephen E. Fienberg. A comparison of string distance metrics for name-matching tasks. In *IJCAI Workshop on Information Integration on the Web*, pages 73–78, 2003.
- [6] Pedro Domingos. Multi-relational record linkage. In *In Proceedings of the KDD-2004 Workshop on Multi-Relational Data Mining*, pages 31–48, 2004.
- [7] Xin Dong, Alon Halevy, and Jayant Madhavan. Reference reconciliation in complex information spaces. In *SIGMOD*, pages 85–96, 2005.
- [8] Steven Euijong Whang, David Menestrina, Georgia Koutrika, Martin Theobald, and Hector Garcia-Molina. Entity resolution with iterative blocking. In *SIGMOD*. ACM, 2009.
- [9] I. P. Fellegi and A. B. Sunter. A theory for record linkage. In *Journal of the American Statistical Society*, volume 64, pages 1183–1210, 1969.
- [10] Full version: <http://www.cs.washington.edu/homes/vibhor/EM09.pdf>.
- [11] Rob Hall, Charles Sutton, and Andrew McCallum. Unsupervised deduplication using cross-field dependencies. In *KDD*, pages 310–317, 2008.
- [12] Dmitri V. Kalashnikov, Sharad Mehrotra, and Zhaoqi Chen. Exploiting relationships for domain-independent data cleaning. In *SIAM International Conference on Data Mining*, 2005.
- [13] Vladimir Kolmogorov and Ramin Zabih. What energy functions can be minimized via graph cuts, 2002.
- [14] Michael Ley. dblp.xml - a documentation: <http://dblp.uni-trier.de/xml/docu/dblp.xml.pdf>.
- [15] Andrew McCallum, Kamal Nigam, and Lyle H. Ungar. Efficient clustering of high-dimensional data sets with application to reference matching. In *Knowledge Discovery and Data Mining*, pages 169–178, 2000.
- [16] Andrew McCallum and Ben Wellner. Conditional models of identity uncertainty with application to noun coreference. In *NIPS*, 2004.
- [17] Gonzalo Navarro. A guided tour to approximate string matching. *ACM Comput. Surv.*, 33(1):31–88, 2001.
- [18] H. B. Newcombe, J. M. Kennedy, S. J. Axford, and A. P. and James. Automatic Linkage of Vital Records. *Science*, 130:954–959, October 1959.
- [19] H. Pasula, B. Marthi, B. Milch, S. Russell, and I. Shpitser. Identity uncertainty and citation matching. In *NIPS*, 2002.
- [20] P Christen R Baxter and T Churches. A comparison of fast blocking methods for record linkage. In *ACM SIGKDD03 Workshop on Data Cleaning, Record Linkage, and Object Consolidation*, 2003.
- [21] Parag Singla and Pedro Domingos. Entity resolution with markov logic. In *icdm*, pages 572–582, 2006.
- [22] Martin J. Wainwright, Tommi S. Jaakkola, and Alan S. Willsky. Tree-reweighted belief propagation algorithms and approximate ML estimation by pseudo-moment matching, 2003.